I. THE CHALLENGE OF LANGUAGE

Unlike an image where all pixels are processed at once, language is **Temporal**. To understand the word "it" in a sentence, the AI must remember the nouns mentioned five words ago. Standard Neural Networks (MLPs) cannot do this because they have no "Memory."

NLP Goals:

- **Sentiment Analysis:** Is this review positive or negative?
- **Machine Translation:** English to Vietnamese.
- **Named Entity Recognition (NER):** Identifying "Duy Tan University" as an Organization.

II. RECURRENT NEURAL NETWORKS (RNN): THE ARCHITECTURE WITH MEMORY

An **RNN** is a type of neural network where the output from the previous step is fed back into the current step as an input. This creates a "Hidden State" (h) that acts as a short-term memory.

2.1. The "Unrolling" Concept

Imagine a loop. For every word in a sentence, the RNN processes the word **AND** the information it remembered from the previous word.

- **Input (x_t):** The current word (converted into a vector/embedding).
- **Hidden State (h_t):** The "Memory" updated at each step.
- **Output (y_t):** The prediction at that specific step.

III. THE LIMITATIONS OF STANDARD RNNs

While the idea is brilliant, standard RNNs suffer from a major mathematical flaw: **The Vanishing Gradient Problem**.

- As the sentence gets longer (e.g., 50 words), the error signal (gradient) during Backpropagation becomes so small that the network "forgets" the beginning of the sentence.
- **Result:** It cannot handle long-term dependencies.

IV. THE SOLUTION: LSTM AND GRU

To fix the memory leak, researchers created "Gated" units that can decide what to remember and what to forget.

4.1. LSTM (Long Short-Term Memory)

The most famous RNN variant. It uses three "Gates":

1. **Forget Gate:** Decides which old information is no longer useful.
2. **Input Gate:** Decides which new information is worth adding to memory.
3. **Output Gate:** Decides what part of the memory to show as the current output.

4.2. GRU (Gated Recurrent Unit)

A simpler, faster version of LSTM. it combines gates to reduce the number of parameters while maintaining similar performance. Great for mobile AI or smaller datasets.

V. WORD EMBEDDINGS: HOW AI "READS"

Computers cannot understand the string "Apple." We must convert words into numbers.

- **One-Hot Encoding:** Simple but inefficient (creates massive, sparse vectors).
- **Word Embeddings (Word2Vec/GloVe):** Maps words into a dense mathematical space where similar words (e.g., "King" and "Queen") are physically close to each other.

VI. PRACTICAL LAB: RNN FOR SENTIMENT ANALYSIS (PYTORCH)

This code shows how to define an LSTM layer in PyTorch to process text.

Python

```
import torch.nn as nn


class SentimentRNN(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, output_dim):
        super(SentimentRNN, self).__init__()
        # 1. Convert word IDs to dense vectors
        self.embedding = nn.Embedding(vocab_size, embed_dim)

        # 2. LSTM Layer (batch_first=True means input is [Batch, Seq, Feature])
        self.lstm = nn.LSTM(embed_dim, hidden_dim, batch_first=True)

        # 3. Final decision layer
        self.fc = nn.Linear(hidden_dim, output_dim)

    def forward(self, text):
        # text shape: [batch_size, sent_length]
        embedded = self.embedding(text)
```

```
# output: all hidden states; hidden: the final 'memory'
output, (hidden, cell) = self.lstm(embedded)

# We only use the last hidden state for classification
return self.fc(hidden[-1])
```